

TECHNICAL REPORT

IoT / EMBEDDED SYSTEMS

Project 5: Local MQTT Broker & Bidirectional Motor / Telemetry Control

Daily Technical Report — Day 14 + Day 15

Platform	ESP32-S3-WROOM · Arduino Framework · PubSubClient
Subsystem	Local Mosquitto MQTT Broker — Bidirectional BME280 Telemetry & L298N Motor Control
Broker	Self-hosted Mosquitto 2.0.22 (WSL 2 / Ubuntu), proxied to the Windows LAN interface
Clients	MQTT Explorer (GUI), mosquitto_pub / mosquitto_sub (CLI), ESP32-S3 firmware
Prepared By	Malak Alshrqawi
Prepared For	IOT Training Program

Confidential — Internal Training Documentation

Contents

1. Daily Overview	3
1.1 Summary	3
1.2 Goal & Submission Format	3
1.3 Tasks Completed Today	3
2. Part 1 — Local Broker Setup on Windows	5
2.1 Why the Default Mosquitto Install Fails Remote Devices	5
2.2 Network Architecture — Windows + WSL 2 + ESP32	5
Key Technical Concepts	5
2.3 Installation & Port Conflict Resolution	5
2.4 mosquitto.conf — Configuration & Rationale	6
2.5 Windows Firewall Rule	6
2.6 WSL 2 NAT Boundary & Port Proxy	7
2.7 CLI Verification — mosquitto_pub / mosquitto_sub	8
2.8 MQTT Explorer Setup	9
3. Part 2 — Topic Structure & Namespace Design	10
Publish — ESP32 → Broker (Telemetry & True State)	10
Subscribe — Broker → ESP32 (Commands)	10
4. Part 3 — Firmware Requirements & Safety Behaviour	10
5. Firmware Architecture	12
5.1 L298N Motor Driver Class	12
5.2 Main Application Flow (setup / loop)	13
5.3 mqttCallback — Command Ingestion	13
5.4 ensureConnections — Reconnect & Failsafe	14
5.5 applyMotorHardware — Failsafe Engine	14
5.6 publishMotorStatus — True-State Reporter	15
6. End-to-End Verification & Screenshots	16
8. References	17

1. Daily Overview

1.1 Summary

This session moved the project from a public/cloud MQTT broker to a fully self-hosted Mosquitto broker running locally on a Windows host inside a WSL 2 (Ubuntu) environment. The objective was to close the remaining gap in the IoT control loop: instead of the ESP32 acting purely as a sensor publisher, it now behaves as a complete bidirectional node — subscribing to command topics that drive an L298N-controlled DC motor, and publishing both live environmental telemetry (temperature, humidity, pressure from a BME280 over I2C) and the motor's true hardware state back to the broker in real time.

Getting a broker installed via apt was the easy part; the real engineering work was making it reachable. Mosquitto v2.0+ binds to the loopback interface by default, and WSL 2's own NAT-isolated virtual subnet sits behind the Windows physical adapter, which meant two separate network boundaries had to be deliberately opened before an ESP32 on the Wi-Fi network could ever see port 1883. This report documents that full path — installation, the address-already-in-use conflict from the auto-started systemd service, the mosquitto.conf changes, the Windows Firewall inbound rule, the netsh portproxy bridge across the WSL NAT boundary, and the final verification using mosquitto_pub/mosquitto_sub before the ESP32 was ever involved — followed by the firmware architecture that makes the motor control loop safe, non-blocking, and self-healing.

1.2 Goal & Submission Format

Goal: run a self-hosted MQTT broker locally and build a complete bidirectional system — control the motor entirely through MQTT topics, while continuously monitoring both environmental data and live motor status coming back from the ESP32.

Submission Format

GitHub repository link + mosquitto.conf + screenshots of MQTT Explorer (connection profile, live topic tree, and publish panel used to drive the motor).

1.3 Tasks Completed Today

- Installed Mosquitto and mosquitto-clients inside WSL 2 (Ubuntu) via apt.
- Diagnosed and resolved an “Error: Address already in use” conflict caused by the auto-started systemd mosquitto service.
- Authored a custom mosquitto.conf binding the listener to 0.0.0.0:1883 and enabling anonymous local connections for prototyping.
- Ran the broker from the terminal in verbose mode (mosquitto -c mosquitto.conf -v) rather than as a background service.
- Mapped the full network path from an external Wi-Fi device to the WSL-hosted broker: Windows Firewall → netsh portproxy → Hyper-V vEthernet NAT switch → WSL 2 virtual subnet.
- Added a Windows Firewall inbound rule for TCP port 1883 via PowerShell (New-NetFirewallRule).
- Bridged the WSL 2 NAT boundary using netsh interface portproxy to forward the Windows LAN IP to the WSL internal IP.
- Verified broker reachability locally with mosquitto_pub / mosquitto_sub in two terminals, then confirmed TCP reachability from the LAN side with Test-NetConnection before touching the ESP32.
- Installed and connected MQTT Explorer natively on Windows to inspect the live topic tree.

- Designed a structured, hierarchical MQTT topic namespace for sensor telemetry and motor command/status.
- Implemented ESP32-S3 firmware: non-blocking BME280 telemetry publishing, MQTT command ingestion with input clamping, true hardware-state feedback, and a network failsafe that stops the motor safely on Wi-Fi/broker loss and auto-recovers on reconnect.

2. Part 1 — Local Broker Setup on Windows

2.1 Why the Default Mosquitto Install Fails Remote Devices

Out of the box, Mosquitto v2.0+ binds strictly to 127.0.0.1 (loopback). Loopback traffic never leaves the local OS instance, so mosquitto_pub/mosquitto_sub running on the same machine succeed while every external device — including an ESP32 on the same Wi-Fi network — is silently refused. On top of that, this deployment runs the broker inside WSL 2, which lives on its own Hyper-V-managed, NAT-isolated virtual subnet. Even after the broker is told to listen on 0.0.0.0, traffic arriving at the Windows host's physical LAN IP still will not reach the WSL virtual IP unless it is explicitly forwarded. Two independent problems therefore had to be solved: (1) get Mosquitto listening on all interfaces, and (2) get Windows to route external traffic across the WSL NAT boundary.

2.2 Network Architecture — Windows + WSL 2 + ESP32

The traffic path for a single MQTT packet leaving the ESP32 and reaching the broker crosses four layers on the Windows host before it ever reaches the Mosquitto process:

Network Path (Wi-Fi device → Mosquitto in WSL)

```
LOCAL WI-FI NETWORK
  ESP32 Node (192.168.1.x)      MQTT Explorer (Windows Host)
    | port 1883                  | port 1883
    v                            v
WINDOWS HOST MACHINE (Physical IP 192.168.1.105)
  1. Windows Firewall inbound rule -> allows TCP/1883
  2. netsh portproxy                -> 192.168.1.105:1883 -> 172.29.160.x:1883
  3. Hyper-V vEthernet (NAT switch) -> bridges Wi-Fi NIC to WSL virtual subnet
    WSL 2 UBUNTU (Internal IP 172.29.160.x)
      4. mosquitto -c mosquitto.conf -v
         listens on 0.0.0.0:1883 across all internal interfaces
```

Key Technical Concepts

Concept	Explanation
Loopback vs. 0.0.0.0	127.0.0.1 (default) never leaves the host; binding the listener to 0.0.0.0 forces Mosquitto to accept connections on every IPv4 interface, including the one reachable from the LAN.
WSL 2 NAT boundary	WSL 2 runs inside a lightweight Hyper-V utility VM with its own isolated virtual subnet (172.29.x.x). Traffic sent to the Windows physical IP is dropped unless a port proxy explicitly forwards it into that subnet.
allow_anonymous true	Mosquitto v2.0+ blocks anonymous connections by default; enabling this allows the ESP32 to publish/subscribe without a username/password store, appropriate for local prototyping.
Listener with no bound IP	Leaving the listener directive without a hardcoded IP binds to 0.0.0.0 automatically and survives DHCP reassigning the host a new address — a hardcoded IP would crash the broker on the next lease renewal.

2.3 Installation & Port Conflict Resolution

Mosquitto and the CLI clients were installed inside the WSL Ubuntu environment:

Bash — Install

```
sudo apt update
sudo apt install mosquitto mosquitto-clients -y
```

The first manual launch (`mosquitto -c mosquitto.conf -v`) failed immediately with Error: Address already in use. The cause: installing Mosquitto via apt on Ubuntu automatically enables and starts it as a background systemd service, which grabs port 1883 on boot. Any attempt to run it manually in the foreground then collides on the same port.

Bash — Stop & Disable the Background Service

```
sudo systemctl stop mosquitto
sudo systemctl disable mosquitto

# Confirm the port is fully free (should return no output)
ss -tuln | grep 1883
```

2.4 mosquitto.conf — Configuration & Rationale

The following minimal, purpose-built configuration file was created in the WSL working directory and launched with `mosquitto -c mosquitto.conf -v` so verbose broker logs remain visible in the terminal at all times, per the “not as a background service” requirement:

mosquitto.conf

```
# Bind port 1883 to 0.0.0.0 (all available network interfaces)
listener 1883

# Allow anonymous clients without username/password for local prototyping
allow_anonymous true
```

Directive	Why it is configured this way
listener 1883 (no IP given)	Omitting the IP causes Mosquitto to bind to 0.0.0.0, i.e. every IPv4 interface on the WSL instance. Hardcoding a specific IP (e.g. 192.168.1.105) would crash the broker with “Cannot assign requested address” whenever DHCP reassigns the host a new lease.
allow_anonymous true	Disables Mosquitto v2.0+'s default requirement for username/password authentication, so the ESP32 and MQTT Explorer can connect immediately during local development without a password file.

Launching in verbose mode confirms the listener is active and logs every client connection, subscription, and disconnection as they occur:

Bash — Run the Broker in the Foreground

```
mosquitto -c mosquitto.conf -v

1785918469: mosquitto version 2.0.22 starting
1785918469: Config loaded from mosquitto.conf.
1785918469: Opening ipv4 listen socket on port 1883.
1785918469: Opening ipv6 listen socket on port 1883.
1785918469: mosquitto version 2.0.22 running
```

2.5 Windows Firewall Rule

Local loopback tests bypass the Windows Firewall entirely, so the broker appeared reachable during same-machine testing even before any firewall rule existed. Any traffic arriving over the physical Wi-Fi adapter (i.e. from the ESP32) is dropped by default until an explicit inbound rule is created. The first attempt failed because New-NetFirewallRule is a PowerShell cmdlet and was run inside cmd.exe, which does not recognise it — PowerShell had to be launched first.

PowerShell — Inbound Firewall Rule for Port 1883

```
powershell
```

```
New-NetFirewallRule -DisplayName "Mosquitto MQTT Broker" -Direction Inbound `
  -LocalPort 1883 -Protocol TCP -Action Allow
```

Change Log — What Was Modified

Windows Defender Firewall: added one Inbound Rule named “Mosquitto MQTT Broker”, Direction = Inbound, Protocol = TCP, LocalPort = 1883, Action = Allow, Profile = Any.

2.6 WSL 2 NAT Boundary & Port Proxy

Even with the firewall rule in place, Test-NetConnection -ComputerName 192.168.1.105 -Port 1883 still reported TcpTestSucceeded : False, despite PingSucceeded being True. This is expected: ping uses ICMP and only confirms the host is powered on and reachable — it says nothing about whether TCP port 1883 is open or forwarded. The real obstacle was the WSL subnet gap: the Windows host sits on 192.168.1.105, while WSL 2 sits on an isolated Hyper-V virtual subnet (172.29.160.x). Windows receives the Wi-Fi traffic on its physical IP but drops it because no rule exists to forward it across the NAT switch into WSL.

Bash — Find the WSL Internal IP (run inside WSL)

```
hostname -I | awk '{print $1}'
# e.g. 172.29.160.2
```

PowerShell (Administrator) — Bridge the NAT Boundary

```
netsh interface portproxy add v4tov4 listenport=1883 listenaddress=192.168.1.105 `
  connectport=1883 connectaddress=<YOUR_WSL_IP>
```

PowerShell — Verify TCP Reachability

```
Test-NetConnection -ComputerName 192.168.1.105 -Port 1883
```

```
ComputerName      : 192.168.1.105
RemoteAddress     : 192.168.1.105
RemotePort        : 1883
TcpTestSucceeded  : True
```

```

C:\Windows\System32\powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\System32> New-NetFirewallRule -DisplayName "Mosquitto MQTT Broker" -Direction Inbound -LocalPort 1883 -Protocol TCP -Action Allow

Name : {fb29c98a-1b64-4a6e-ba1c-860165cf5a95}
DisplayName : Mosquitto MQTT Broker
Description :
DisplayGroup :
Group :
Enabled : True
Profile : Any
Platform : ()
Direction : Inbound
Action : Allow
EdgeTraversalPolicy : Black
LooseSourceMapping : False
LocalOnlyMapping : False
Owner :
PrimaryStatus : OK
Status : The rule was parsed successfully from the store. (65536)
EnforcementStatus : NotApplicable
PolicyStoreSource : PersistentStore
PolicyStoreSourceType : Local
RemotelyDynamicKeywordAddresses : {}
PolicyId :

```

Step 5: WSL 2 NAT Networking & Port Proxy Setup

```

PS C:\Users\cyber> wsl
malak@cyber:/mnt/c/Users/cyber$ mosquitto_pub -h 127.0.0.1 -p 1883 -t "lab/test" -m "Mosquitto is live!"
malak@cyber:/mnt/c/Users/cyber$ nc -zv 192.168.1.15 1883
nc: connect to 192.168.1.15 port 1883 (tcp) failed: Connection refused
malak@cyber:/mnt/c/Users/cyber$ nc -zv 127.1.1.15 1883
Connection to 127.1.1.15 1883 port [tcp/*] succeeded!
malak@cyber:/mnt/c/Users/cyber$ exit
logout
PS C:\Users\cyber> Test-NetConnection -ComputerName 192.168.1.15 -Port 1883
WARNING: TCP connect to (192.168.1.15 : 1883) failed

ComputerName : 192.168.1.15
RemoteAddress : 192.168.1.15
RemotePort : 1883
InterfaceAlias : Wi-Fi
SourceAddress : 192.168.1.105
PingSucceeded : True
PingReplyDetails (RTT) : 19 ms
TcpTestSucceeded : False

PS C:\Users\cyber> Test-NetConnection -ComputerName 127.0.0.1 -Port 1883

ComputerName : 127.0.0.1
RemoteAddress : 127.0.0.1
RemotePort : 1883
InterfaceAlias : Loopback Pseudo-Interface 1
SourceAddress : 127.0.0.1
TcpTestSucceeded : True

```

Figure 1 — WSL 2 NAT diagnostics: `mosquitto_pub/nc` loopback tests succeed, direct LAN-IP connection fails, then succeeds after the portproxy bridge is added.

2.7 CLI Verification — `mosquitto_pub` / `mosquitto_sub`

Before the ESP32 was involved at all, the broker was exercised end-to-end from two separate terminals: one publishing motor command topics, the other subscribing to live sensor and motor status topics, to confirm the full pub/sub path and topic routing worked correctly.

Terminal A — `mosquitto_sub` (subscriber)

```

mosquitto_sub -h 127.0.0.1 -p 1883 -t "esp32/#" -v
esp32/sensor/temperature 22.91
esp32/sensor/humidity 44.94
esp32/sensor/pressure 892.00
esp32/motor/speed 50
esp32/motor/direction backward
esp32/motor/state running

```

Terminal B — `mosquitto_pub` (publisher, motor commands)

```

mosquitto_pub -h 127.0.0.1 -p 1883 -t "esp32/motor/cmd/power" -m "on"
mosquitto_pub -h 127.0.0.1 -p 1883 -t "esp32/motor/cmd/direction" -m "forward"

```

```
mosquitto_pub -h 127.0.0.1 -p 1883 -t "esp32/motor/cmd/speed" -m "75"
```

2.8 MQTT Explorer Setup

MQTT Explorer was installed natively on Windows (not inside WSL) because a native GUI on the host exercises the exact same network path a Wi-Fi client such as the ESP32 must take — through the physical adapter, the firewall rule, and the portproxy bridge — rather than the loopback shortcut a WSL-hosted client would use.

Field	Value
Protocol	mqtt://
Host	192.168.1.105 (Windows LAN IP)
Port	1883
Username / Password	blank (allow_anonymous true)

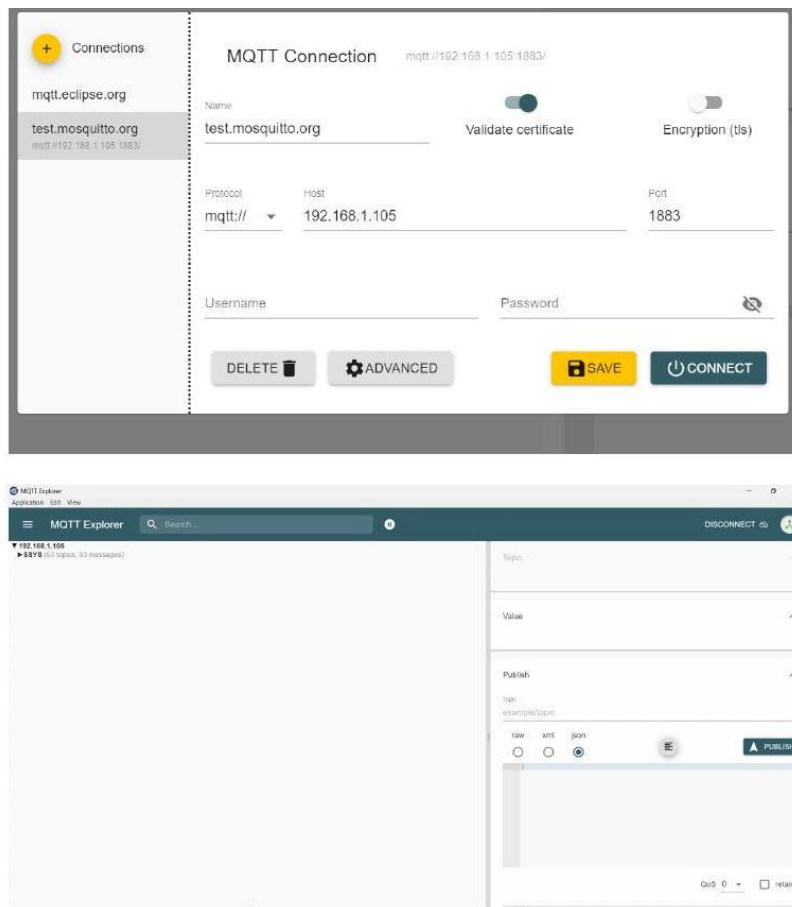


Figure 2 — MQTT Explorer connection profile pointed at the Windows LAN IP, and the resulting live topic tree (\$SYS system topics confirming a successful client handshake).

3. Part 2 — Topic Structure & Namespace Design

The topic hierarchy follows a structured, non-flat namespace rooted at esp32/, split cleanly into a sensor branch (device → broker) and a motor branch, which itself separates true device state (device → broker) from incoming commands (broker → device). This keeps status and control unambiguous: nothing that the ESP32 publishes as its own state can be confused with something a client is trying to command it to do.

Publish — ESP32 → Broker (Telemetry & True State)

Topic	Payload	Meaning
esp32/sensor/temperature	float, °C	BME280 temperature reading
esp32/sensor/humidity	float, %RH	BME280 relative humidity reading
esp32/sensor/pressure	float, hPa	BME280 barometric pressure reading
esp32/motor/state	"running" "stopped"	Actual motor state, derived from live driver output
esp32/motor/direction	"forward" "backward" "stopped"	Actual direction currently applied to the H-Bridge
esp32/motor/speed	integer 0-100	Actual speed, converted back from raw PWM (0-255) to a percentage

Subscribe — Broker → ESP32 (Commands)

Topic	Payload	Meaning
esp32/motor/power	"on"/"1"/"ON" or "off"/"0"	Motor ON/OFF setpoint
esp32/motor/direction	"forward" "backward"	Direction setpoint
esp32/motor/speed	integer, clamped 0-100	Speed setpoint as a percentage

The ESP32 subscribes to the wildcard esp32/motor/# so any new command sub-topic is picked up automatically without a firmware change, while telemetry and true-state topics are published individually so MQTT Explorer and any other subscriber can watch each value change independently.

4. Part 3 — Firmware Requirements & Safety Behaviour

Requirement	How it is satisfied
Non-blocking sensor timing	loop() compares millis() against a stored lastMsgTime on every pass; BME280 values are read and published only once the configured 2000 ms interval has elapsed, with no delay() blocking the rest of the loop.
Motor status reflects real state, not last command	publishMotorStatus() never echoes the last received command. It queries motor.getSpeed() / motor.getDirection() directly from the L298N driver instance — the true, currently-applied hardware output — and publishes that.
Speed validation & clamping	Every incoming esp32/motor/speed payload is parsed with message.toInt() and passed through constrain(SpeedValue, 0, 100) before it is ever stored in targetSpeedPer, so out-of-range or malformed values cannot reach the motor driver.

Reconnect handling — no crash, safe stop	<code>ensureConnections()</code> checks <code>WiFi.status()</code> and <code>client.connected()</code> on every loop iteration. On any disconnect it immediately forces <code>motor.stop()</code> through <code>applyMotorHardware()</code> , then calls <code>WiFi.reconnect()</code> (or re-subscribes to the broker) inside a bounded, non-blocking retry window instead of looping forever.
--	---

5. Firmware Architecture

The firmware keeps the same modular, object-oriented structure used in earlier projects: an L298N class encapsulates all low-level pin driving and state tracking for the motor driver hardware, and the main application file owns networking, MQTT command parsing, sensor polling, and a dedicated failsafe/state-reporting layer that sits between MQTT and the hardware.

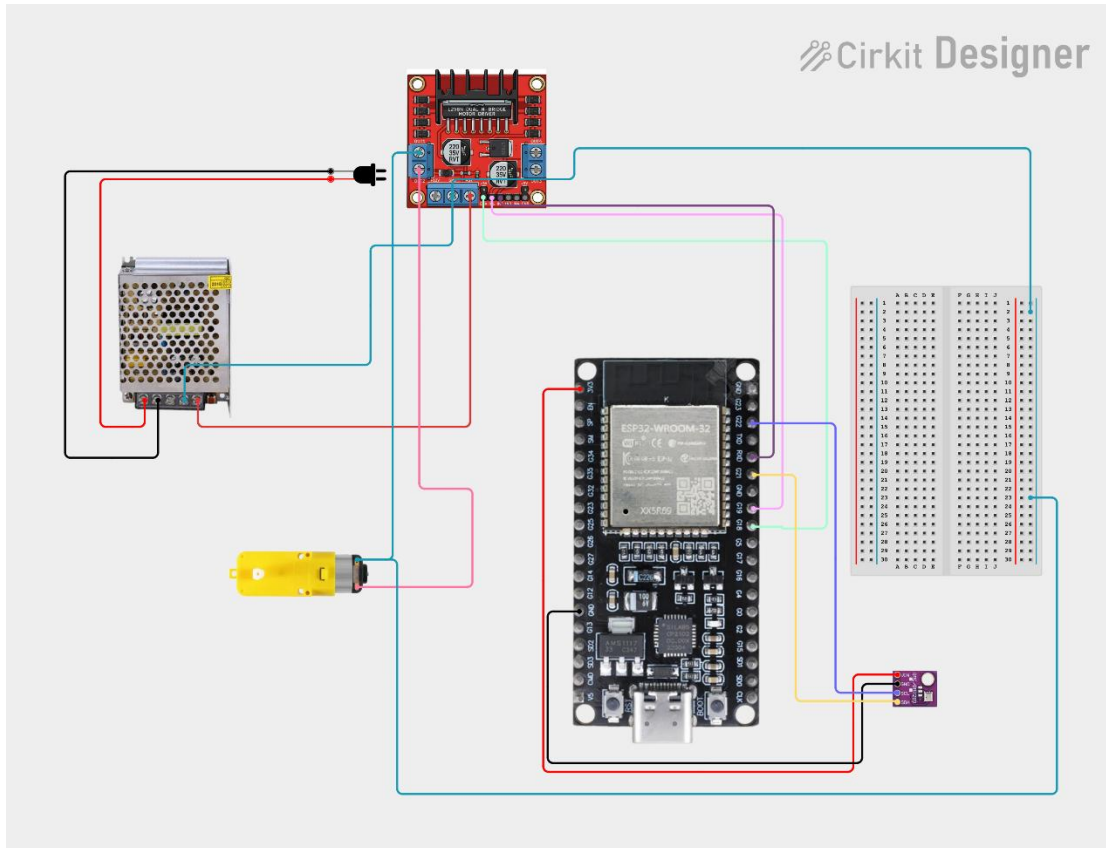


Figure 0 — Hardware wiring: ESP32-S3-WROOM, L298N H-Bridge driving a DC gear motor, an IR obstacle sensor, and a breadboard-mounted BME280/LED test node, powered from a bench PSU.

5.1 L298N Motor Driver Class

Method	Behaviour
L298N(pin1, pin2, ena)	Constructor. Stores GPIO assignments for direction line 1, direction line 2, and the PWM speed-enable pin; initialises internal speed and direction state to 0.
begin()	Configures all three pins as OUTPUT and immediately calls stop() so the driver outputs are guaranteed LOW at boot.
forward(speed)	Sets <code>_currentspeed / _currentdirection = 1</code> , drives PWM to the enable pin, pulls pin1 HIGH and pin2 LOW.
reverse(speed)	Sets <code>_currentspeed / _currentdirection = 2</code> , drives PWM to the enable pin, pulls pin1 LOW and pin2 HIGH.

stop()	Resets internal state to 0/0, sets PWM to 0, and pulls both direction pins LOW — completely isolating motor power so current cannot drift or the coils overheat.
Brake()	Applies active electronic braking: full PWM (255) with both direction pins pulled HIGH simultaneously.
getSpeed() / getDirection()	Getters returning the raw internal PWM (0-255) and direction code (0 = stopped, 1 = forward, 2 = backward) — the single source of truth used for MQTT status feedback.

5.2 Main Application Flow (setup / loop)

setup() starts serial diagnostics, initialises the motor driver, brings up I2C and probes both possible BME280 addresses (0x76 and 0x77), registers Wi-Fi credentials, configures the MQTT broker target and callback, then blocks briefly for the initial Wi-Fi handshake.

setup()

```
void setup() {
  Serial.begin(115200);
  motor.begin();
  Wire.begin(SDA_PIN, SCL_PIN);
  if (!bme.begin(0x76)) { if (!bme.begin(0x77)) Serial.println("BME280 not found"); }
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  client.setServer(MQTT_BROKER, MQTT_PORT);
  client.setCallback(mqttCallback);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
}
```

loop() is the non-blocking heart of the system: it services the network stack every pass (ensureConnections(), client.loop()), publishes BME280 telemetry only when the millis()-based interval has elapsed, and otherwise keeps applying the current target motor state through applyMotorHardware() so the driver output always reflects the latest validated command.

loop() — condensed

```
void loop() {
  ensureConnections();
  client.loop();
  currentMillis = millis();
  if (currentMillis - lastMsgTime >= msgInterval) {
    lastMsgTime = currentMillis;
    if (WiFi.status() == WL_CONNECTED && client.connected()) {
      // read BME280, client.publish() temperature / humidity / pressure
    }
  } else {
    applyMotorHardware(motor, targetDirection, targetSpeedPer, targetPower, client);
    delay(100);
  }
}
```

5.3 mqttCallback — Command Ingestion

Every inbound message is reassembled into a trimmed String and routed by topic. Power is parsed as a boolean from several accepted literals; direction is only accepted if it matches an explicit "forward"/"backward" value; speed is parsed as an integer and immediately clamped with constrain(SpeedValue, 0, 100) before being stored. The callback

finishes by re-applying the hardware state and publishing fresh status, so a single command always produces an immediate, matching status update.

mqttCallback(char *topic, byte *payload, unsigned int length) — condensed

```
if (topicStr == "esp32/motor/power")
    targetPower = (message == "1" || message == "on" || message == "ON");
else if (topicStr == "esp32/motor/direction") {
    if (message == "forward" || message == "backward") targetDirection = message;
}
else if (topicStr == "esp32/motor/speed") {
    SpeedValue = message.toInt();
    targetSpeedPer = constrain(SpeedValue, 0, 100); // hard clamp, always 0-100
}
applyMotorHardware(motor, targetDirection, targetSpeedPer, targetPower, client);
publishMotorStatus(motor, client);
```

5.4 ensureConnections — Reconnect & Failsafe

This function is the system's self-healing layer. If Wi-Fi has dropped, it forces an immediate `motor.stop()` via `applyMotorHardware()` before ever attempting to reconnect, then calls `WiFi.reconnect()` and waits inside a bounded 10-second window rather than blocking indefinitely. If the MQTT session itself has dropped while Wi-Fi is fine, it likewise forces a stop, reconnects to the broker as "ESP32Client", re-subscribes to `esp32/motor/#`, and republishes the current hardware state so any client reconnecting to the broker immediately sees accurate status.

ensureConnections() — condensed

```
if (WiFi.status() != WL_CONNECTED) {
    applyMotorHardware(motor, targetDirection, targetSpeedPer, targetPower, client); // stop first
    WiFi.reconnect();
    currentMillis = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - currentMillis < 10000) delay(500);
}
if (WiFi.status() == WL_CONNECTED && !client.connected()) {
    applyMotorHardware(motor, targetDirection, targetSpeedPer, targetPower, client); // stop first
    client.connect("ESP32Client");
    if (client.connected()) { client.subscribe("esp32/motor/#"); publishMotorStatus(motor, client); }
}
```

5.5 applyMotorHardware — Failsafe Engine

This is the single choke point through which every physical motor output must pass. It refuses to drive the motor at all — forcing `motor.stop()` — if Wi-Fi is down, if the MQTT client is disconnected, or if `targetPower` is false. Only when all three checks pass does it convert the validated 0-100% setpoint to an 8-bit PWM value (0-255) via `map()` and invoke `motor.forward()` or `motor.reverse()`.

applyMotorHardware(...) — condensed

```
if (WiFi.status() != WL_CONNECTED || !client.connected() || !targetPower) {
    motor.stop();
    return;
}
speed = map(targetSpeedPer, 0, 100, 0, 255);
if (targetDirection == "forward") motor.forward(speed);
else if (targetDirection == "backward") motor.reverse(speed);
```

```
else motor.stop();
```

5.6 publishMotorStatus — True-State Reporter

Rather than echoing the last received command, this function reads the live raw PWM and direction codes directly back out of the motor instance, converts the raw PWM back to a 0-100% figure, derives a running/stopped state from the actual values, and publishes all three to esp32/motor/speed, esp32/motor/direction, and esp32/motor/state — guaranteeing the values seen in MQTT Explorer always match the motor's real, physical behaviour.

publishMotorStatus(...) — condensed

```
speed = motor.getSpeed(); direction = motor.getDirection();
dirstr = (direction == 1) ? "forward" : (direction == 2) ? "backward" : "stopped";
speed = map(speed, 0, 255, 0, 100);
stateStr = (speed > 0 && direction != 0) ? "running" : "stopped";
client.publish("esp32/motor/speed", String(speed).c_str());
client.publish("esp32/motor/direction", dirstr.c_str());
client.publish("esp32/motor/state", stateStr.c_str());
```

6. End-to-End Verification & Screenshots

With the broker running verbosely from the terminal, the ESP32 flashed and connected over Wi-Fi to the Windows LAN IP, and MQTT Explorer pointed at the same address, the full loop was exercised: temperature/humidity/pressure updating continuously on their own topics, and power/direction/speed commands published from MQTT Explorer producing an immediate, correct change in the esp32/motor/* status topics.

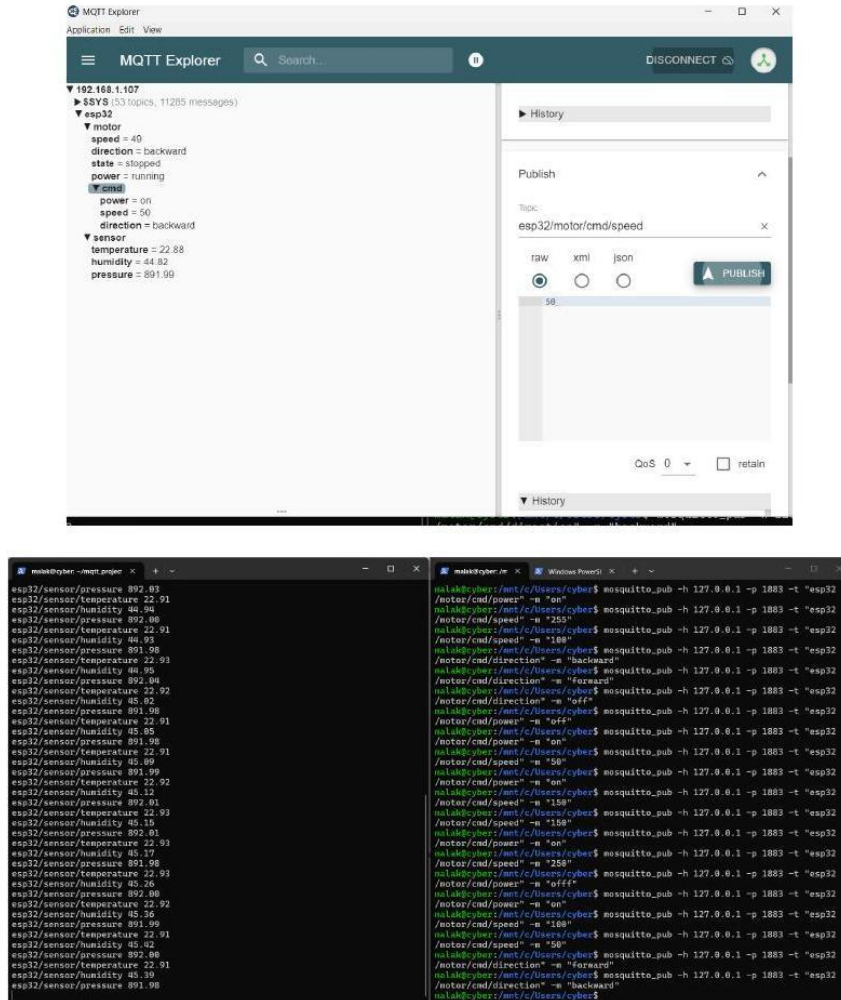


Figure 3 — MQTT Explorer live topic tree (esp32/motor/speed=49, direction=backward, state=stopped, power=running, plus the esp32/motor/cmd/* command topics) alongside two terminals: one streaming continuous BME280 telemetry, the other issuing mosquitto_pub power/speed/direction commands against the broker.

Observed behaviour during testing matched the design intent in every case: publishing esp32/motor/cmd/power = off or stopping the broker/killing Wi-Fi caused the motor to stop immediately and the esp32/motor/state topic to report "stopped" on the very next status publish — never left stale on the last "running" value. Restarting the broker or restoring Wi-Fi resulted in automatic reconnection, re-subscription, and a fresh status publish, all without requiring an ESP32 reset.

8. References

Mosquitto MQTT Broker: Pros, Cons, Tutorial and Modern Alternatives — EMQX.

<https://www.emqx.com/en/blog/mosquitto-mqtt-broker-pros-cons-tutorial-and-modern-alternatives>

MQTT Explorer — <https://mqtt-explorer.com>

Eclipse Mosquitto documentation — `mosquitto.conf` reference, listener and `allow_anonymous` directives.